Ders 6 Genişletilmiş Notu: Test Yönetimi, Test Planı ve Yazılım Bakımı

Ders 6 Genişletilmiş Notu: Test Yönetimi, Test Planı ve Yazılım Bakımı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Gereksinim Spesifikasyonu Formatı Üzerine Soru-Cevap
- 2. Test Süreci Adımlarının Kısa Tekrarı
- 3. Yazılım Ölçümü Neden Gerekir?
- 4. Halstead ve McCabe Metrikleri
- 5. Yazılım Güvenirliği
- 6. Büyüklük Ölçütleri: Function Point ve Kod Satırı
- 7. CK Metriklerinin Tekrarı
- 8. Dersin O Güne Kadarki Akışı ve Test Tipleri
- 9. Test Yönetimi ve Test Spesifikasyonu
- 10. Test Planının İçeriği
- 11. Test Senaryosu
- 12. Test Çalışma Ekibi ve Optimum Test Miktarı
- 13. Yazılım Yaşam Döngüsü Boyunca Test Etkinlikleri
- 14. Teslim Sonrası Kalite: Yazılım Bakımı
- 15. Yazılım Konfigürasyon Yönetimi

1. Gereksinim Spesifikasyonu Formatı Üzerine Soru-Cevap

Ders, dönem projesinin ilk adımı olan gereksinim analizi ve gereksinim spesifikasyonu raporu hakkındaki sorularla başladı. Bu kısımdaki akademik içerik, gereksinim belgesinin hangi biçimde yazılacağıyla ilgilidir.

Soru: Gereksinim belgesi için verilen IEEE formatının başlıklarına sıkı sıkıya bağlı kalmak zorunda mıyız, yoksa biraz esnetebilir miyiz?

Cevap: Hayır, zorunlu değil. Hoca bu formatı yalnızca yol gösterici olarak verdiğini söyledi. Bu form, standartlara sıkı bağlı çalışan projeler için geçerlidir. Özellikle ilk bölümlerin ikinci ve üçüncü maddelerinde çok fazla alt başlık vardır; bunlar genelleştirilip özetlenebilir. Derste daha önce gösterilen, başka bir üniversitede yapılmış eski bir öğrenci projesinin gereksinim spesifikasyonu da zaten özetlenmiş bir örnekti.

Hoca ayrıca fonksiyonel spesifikasyonların yazımında serbest olunduğunu hatırlattı: Gereksinimler madde madde listelenebilir ya da kullanım durumu (use case) belgeleriyle gösterilebilir. İkisi de geçerlidir.

> **Ders sonrası not:** Soruyu soran öğrenci formatı “IEEE 802” diye andı, hoca da “evet” dedi. IEEE 802 büyük olasılıkla yerel ve metropol ağ standartlarıdır. Gereksinim spesifikasyonu formatı IEEE 830’dur; hoca dersin ilerisinde (bkz. Bölüm 9) 830 numarasını gereksinim spesifikasyonu standardı olarak doğru biçimde andı.

2. Test Süreci Adımlarının Kısa Tekrarı

Hoca her zamanki kuralı bozmadan bir önceki haftayı özetleyeceğini, ancak test süreci adımları üzerinde artık fazla durmayacağını söyledi; bunlar önceki haftalarda yeterince işlenmişti. Yine de bir öğrenciye sorarak kontrol etti.

Soru: Bir yazılımın kodlaması tamamlandı. Test süreci adımlarında sırasıyla hangi testleri gerçekleştiririz?

Hoca, soruyu "testin planlanması ve tasarımı" diye yanıtlayan bir cevaba karşılık bunun sorulmadığını belirtti: Test planlaması ve tasarımı test yönetiminin konusudur ve o konuya henüz gelinmemişti. Soru, hangi testlerin hangi sırayla yapıldığıdır.

Cevap: Birim testi (unit test), bütünlük testi (integration test), sistem testi ve ardından kullanıcı testi.

Hoca bundan sonra test süreci adımları ve test tipleri üzerinde durmayacağını, yalnızca geçen hafta görülen ölçüm sistemleri konusunu tekrar edeceğini söyledi.

3. Yazılım Ölçümü Neden Gerekir?

Yazılım ölçümü (software metrics) yazılım mühendisliğinin önemli konularından biridir. Hocaya göre yazılım geliştirme sürecinde yaptığımız işleri ve sonuçta elde ettiğimiz ürünü ölçebilirsek, yazılım geliştirme sürecini daha iyi yönetebilir ve kontrol edebiliriz.

Yöneticilerin ve yazılım mühendislerinin ölçümden beklentileri farklıdır. Aslında aynı şeyleri isterler, ama farklı açılardan bakarlar.

Yöneticiler ölçme işlemini şunlar için ister:

- Yazılım üretiminin maliyetinin ne olacağını görmek.
- Farklı bölüm çalışanlarının ücretlerini belirlemek. Hangi yazılımın hangi aşamasında ne kadar iş gücü harcandığı ve ne büyüklükte bir yazılım üretildiği bilinirse personelin verimliliği de ölçülebilir.
- Kullanılan altyapı yazılım ürünlerini ve geliştirme araçlarını kıyaslamak.

Yazılım mühendisleri ise şunlar için ister:

- Ürünün ne büyüklükte bir yazılım olduğunu ölçmek.
- Süreç ölçümleriyle sistemin kalitesini daha iyi izlemek: tasarım sürecinde yapılan değişiklikleri ve test adımlarında bulunan hataları görmek.
- Performans ve kalite gereksinimlerini belirlemek.
- Sertifika almak: Hem ürüne hem de yazılım evine sertifika alabilmek için ürün ve süreç özellikleri ölçülür. Hocanın örneği, modüllerin yüz kod satırından fazla olmamasına dikkat etmek gibi bir ölçüttü.
- Gelecekteki ürün geliştirmelerinde tasarım yollarını daha iyi belirlemek.

Kısacası ölçümün iki amacı vardır: Şu anda geliştirilen yazılımın performans ve verimlilik özelliklerini yönetmek ve elde bir arşiv verisi oluşturmak. Bu arşiv sayesinde sonraki yazılımlarda büyüklük, iş gücü ve diğer özellikler kıyaslanabilir.

4. Halstead ve McCabe Metrikleri

Hoca geçen hafta söz edilen metrikleri tek tek hatırlattı. Algoritma karmaşıklığı da bir ölçüttür, ama algoritma derslerinde yeterince işlendiği için üzerinde durulmadı.

4.1 Halstead Metriği

Bir yazılımın kodu uzun olabilir ama içinde çok az işlem geçebilir; yalnızca düz akışlı okuma ve yazdırma işlemleriyle yürüyebilir. Tersine, kısa bir kod çok fazla hesaplama içerebilir. Halstead metriği bu farkı yakalamaya çalışır. Şu sorulara bakılır:

- Hesaplama işlemi ne kadar çok?
- Bu işlemlere giren eleman, yani işlenen (operand) sayısı ne kadar fazla?
- Bu işlemler ne kadar döngü içinde kullanılıyor?

Bunlar dikkate alınarak program uzunluğu, program hacmi ve program gücü gibi değerler hesaplanır.

4.2 McCabe Döngüsel Karmaşıklığı

McCabe metriği programın tasarım aşamasında kullanılır ve programın kontrol akışı hakkında bilgi verir. Kontrol akışı, döngüleri ve if bloklarını yönlendiren kontrol deyimleriyle belirlenir. Bu deyimler bir akış grafında (program grafı) düğümler ve kenarlar olarak gösterilir. Önce bu çizim yapılır, karmaşıklık derecesi de bunun üzerinde hesaplanır.

İngilizcedeki cyclomatic complexity teriminin karşılığı olarak döngüsel karmaşıklık kullanılır. Hesaplamanın derste anlatılan yolu şudur: Akış grafı çizildikten sonra graf içindeki kapalı bölgeler sayılır ve buna grafın dışında kalan bölge de eklenir. Elde edilen sayı modülün karmaşıklığını verir.

Hoca bunu Pressman'ın kitabından aldığı bir şekil üzerinde gösterdi. Şekilde ortalama alma algoritmasından çıkarılmış, 1, 2, 3, 4, 5, 6, 7, 8 gibi numaralı düğümleri ve kenarları olan bir akış grafı vardı. Bölgeler şöyleydi:

- R_1: en dışta kalan bölge.
- R_2, R_3, R_4, R_5, R_6: kapalı bölgeler.

Bu bölgeler sayılarak döngüsel karmaşıklık değeri ve programda kaç tane bağımsız yol olduğu görülür. Bu örnekte bölge sayısı 6'dır, dolayısıyla döngüsel karmaşıklık $V(G) = 6$ olur (ara adım eklendi).

McCabe ölçüsü en yüksek modül büyüklüğünü saptamaya da yardımcı olur. Modülün döngüsel karmaşıklığının en çok 10 olması gerektiği yönünde bir genelleme yapılmıştır. Yani bir algoritmadaki bağımsız yol sayısı 10'dan fazla olursa modül çok karmaşık olur ve test edilmesi zorlaşır. Bu durumda modülü daha alt parçalara bölmek iyi olur. Döngüsel karmaşıklığın fazla olması, bir modülün test edilmesini zorlaştıran bir etkendir.

> **Ek bilgi (derste anlatılmadı):** Döngüsel karmaşıklık, bölge saymanın yanında büyük olasılıkla iki eşdeğer formülle de hesaplanır: $V(G) = E - N + 2$ (E kenar, N düğüm sayısı) ve $V(G) = P + 1$ (P karar düğümü sayısı). Pressman'daki ortalama alma örneği büyük olasılıkla 17 kenar ve 13 düğümlü grafiktir; bu da $17 - 13 + 2 = 6$ verir. Bağımsız yol sayısı büyük olasılıkla saydam kutu testinde temel yol testi (basis path testing) için gereken en az test durumu sayısıdır.

5. Yazılım Güvenirliği

Hocaya göre tüm yazılımların aynı kalitede olması ve aynı bileşenlerle aynı sonucu vermesi beklenen bir durum değildir. Bir yazılımda mutlaka hata kalacaktır. Güvenirlik (reliability) bu hatanın miktarıyla ilişkilidir. Güvenirlik değeri geçmiş bilgilere ve teste dayanarak belirlenir. Örneğin yazılımın toplam işletim süresinin ne kadarında başarılı olarak çalıştığı kestirilir. Yani yazılım güvenirliği bir kestirime dayalı çalışmadır.

Yazılım güvenirliği modelleri daha önceki kullanımlarda tutulmuş istatistiklere dayanarak güvenirliği saptar. Bu modellerle şu değerler elde edilir:

- Toplam hata sayısı.
- Güvenirlik düzeyi.
- Sistemde yapılan revizyon sonucunda kalan hata sayısı.

1980'lerden beri yüze yakın güvenirlik modeli geliştirilmiştir; hoca Jelinski modelini örnek verdi. Bu modeller hâlâ yazılım mühendisliğinde önemlidir. Hocanın ifadesiyle soru şudur: Yazılım doğru bir sonuç veriyor, ama bu sonuca ne kadar güvenebiliyoruz?

Yüze yakın model arasından seçim yaparken şu sorular sorulur:

- Bu modele göre yazılım güvenirliğini ne kadar kolay ölçebiliyoruz?
- Ölçerken yazılımın tümünü kullanmamız gerekiyor mu?
- Model kolay ve anlaşılır biçimde kullanılabiliyor mu?
- Model dış etkenlerden etkileniyor mu? Yani bulunan sonuçlar yazılımın ortamındaki nelerden etkileniyor?

5.1 Güvenirliği Sağlamak İçin Yaşam Döngüsü Boyunca Yapılanlar

Hoca güvenirliği sağlamak için her aşamada yapılması gerekenleri sıraladı:

- **Gereksinim sırasında:** Yazılımın işlevsel yapısını iyi belirlemek, hataları tanımlamak ve sınıflandırmak. Önceki derslerde hatalar büyüklüğüne ve önemine göre sınıflandırılmıştı: ölümcül hatalar, büyük hatalar, orta hatalar gibi. Ayrıca müşterinin güvenirlik ihtiyaçlarını, yani güvenirlik anlamındaki beklentisini iyi tanımlamak gerekir.
- **Tasarım ve uygulama sırasında:** Bileşenler arasındaki güvenirliği belirlemek, işlevsel kaynaklara odaklanmak, birim modüller yazılırken hatalı kodlamaları incelemek, birim test sırasında yapılacakları kodlama sırasında gözetmek ve güvenirliği ölçmek.
- **Sistem testi sırasında:** Sistemin bütünü üzerinde güvenirlikle ilgili testleri yapmak; test yönetiminde ihtiyaç duyulan ek testler varsa bunları belirlemek. Test edilen tek tek yazılım elemanlarının güvenirliği de geçerlenmelidir.
- **Teslim sırasında:** Yazılım kullanıma sunulurken güvenirliği tekrar kontrol etmek ve müşteri memnuniyetini izlemek.

6. Büyüklük Ölçütleri: Function Point ve Kod Satırı

Hoca slaytta olmayan, ama daha önce üzerinde durulan büyüklük ölçütlerine geçti ve bir öğrenciye sordu.

Soru: Bir yazılım var. Fiziksel olarak ne kadar büyük olduğunu genel olarak hangi yollarla ölçebiliriz?

Öğrenci veritabanına veya fonksiyonların sayısına göre ölçülebileceğini söyledi. Hoca veritabanının başka bir büyüklük ölçütünün içinde geçtiğini açıkladı: Veritabanında kaç dosya olduğu, kaç girdi ve kaç çıktı olduğu sayılır. Bu dolaylı yolla yapılan bir ölçümdür. Hoca soruyu sürdürdü: Veritabanı, girdi-çıktı (input-output) değerleri ve ara birim değerleri sayılıp bunlar üzerinden hesaplama yapılan büyüklük ölçümü hangisidir?

Cevap: İşlev noktası (function point). Hoca bunun bir karmaşıklık ölçüsü değil, büyüklük için dolaylı bir ölçüm yolu olduğunu vurguladı. Karmaşıklık ölçüsü McCabe ölçütünde görülmüştü.

Bir başka büyüklük ölçütü de doğrudan ölçümdür: kod satırı (line of code, LOC), yani kaç kod satırlık bir yazılım hazırlandığı. Birim olarak bin kod satırı (KLOC) da kullanılır.

Soru: LOC'u kullanmak çok basittir, sadece kodu sayarız. Peki kötü tarafları, dezavantajları nelerdir?

Cevap: Bir öğrenci şu örneği verdi: Örneğin bir print yazdık; ama başlangıç parantezini ve kapatma parantezini ayrı satırlara, araya boş satırlar bırakarak yazarsak satır sayısı artar. Hoca bu cevabı onayladı ve dezavantajları şöyle topladı:

- **Yazım biçimi ve programlama dili:** Kodun işlevi değil, kod sayısı değişir. C dilinde çok satırda yazılan bir şey başka bir dilde daha az satır tutabilir; Assembler'da ise daha fazla tutar. Bu farkı ortadan kaldırmak için, hangi dildeki kaç satırın hangi dilde kaç satıra denk geldiğini gösteren bir eşleştirme tablosu hazırlanmıştır.
- **Algoritma:** Etkin bir programlama yaparsak aynı şeyi daha az satırla anlatabiliriz. Daha çok uğraşıp daha iyi bir algoritma tasarlayan, daha az satırda kod yazar.
- **Yeniden kullanım (reusability):** Yeniden kullanılan kod da satır sayısını düşürür.

Bu nedenlerle programcının harcadığı vakit ve iş gücü satır sayısıyla her zaman doğru orantılı değildir. Bu da satır sayısının büyüklük ölçütü olması konusunda bir çelişki yaratır.

> **Hocanın vurgusu:** LOC ölçmesi çok kolay olduğu için birçok şeyde yine de baz alınır. Bu dezavantajlar bilinerek dikkate alınır.

7. CK Metriklerinin Tekrarı

Hoca nesneye yönelik yazılımlar için kullanılan ölçüm sistemlerinin CK metrikleri olarak özetlendiğini hatırlattı. Bu ad, konu üzerinde çalışan Chidamber ve Kemerer'den gelir. Aradan sonra kümedeki ölçütlerin üzerinden tek tek geçti.

7.1 Sınıfın Ağırlıklı Metot Sayısı (WMC)

Bir sınıfın tüm metotlarının karmaşıklıklarının toplamıdır. Buradaki karmaşıklık değeri döngüsel karmaşıklık da olabilir, algoritma karmaşıklığı da. Bu değer hem yazılım kalite ölçütü olarak hem de yazılımın ne kadar emek harcanarak geliştirildiğini göstermesi bakımından önemlidir. Sınıflardaki metot sayısı da yazılımın bir ölçütü olarak kullanılır.

7.2 Kalıtım Ağacı Derinliği (DIT)

Sınıfın kalıtım ağacının köküne olan uzaklığıdır. Hiçbir sınıftan türetilmemiş bir sınıf için bu değer sıfırdır. Bazı sınıflarda çoklu kalıtım olabilir; bu durumda en uzak köke olan uzaklık alınır.

Kalıtım hiyerarşisinde daha derinde olan sınıflar daha çok metot devraldıkları için davranışlarını tahmin etmek zordur. Bu yüzden derin bir kalıtım ağacındaki sınıfın tasarım karmaşıklığı daha fazladır.

7.3 Alt Sınıf Sayısı (NOC)

Alt sınıf sayısı, yani çocuk sayısı (number of children), sınıftan doğrudan türetilmiş alt sınıfların sayısıdır. Bir sınıfın çok fazla alt sınıfı varsa bu, kalıtımın yanlış kullanıldığının bir göstergesi olabilir. Çok alt sınıfı olan sınıfların metotları daha çok test gerektirir. Bu metrik, sınıfı test etmek için harcanacak bütçeyi de artırır.

7.4 Nesne Sınıfları Arasındaki Bağımlılık (CBO)

Coupling between object classes, sınıfın bağımlı olduğu sınıf sayısıdır. Bir sınıfın nitelikleri ya da metotları başka bir sınıfta kullanılıyorsa ve iki sınıf arasında kalıtım yoksa bu iki sınıf arasında bağımlılık vardır.

- Aşırı bağımlılık modüler tasarıma zarar verir. Bu yüzden kalite açısından tasarım sırasında bağımlılığın çok fazla olmamasına dikkat eden bir modül tasarımı yapılır.
- Bağımlılıktaki artış değişime duyarlılığı da artırır; yazılım bakımı zorlaşır.
- Bağımlılık, tasarımın farklı parçalarının ne kadar karmaşık test edileceği hakkında da fikir verir. Bağımlılığın fazla olması test maliyetini artırır.

Hoca, dengede olması için bağımlılığın ne kadar olması gerektiğinin tasarım planlamasında belirlenebileceğini ekledi.

7.5 Sınıfın Tetiklediği Metot Sayısı (RFC)

Response for class, verilen sınıftan bir nesnenin metotları çağrıldığında bu nesnenin tetikleyebileceği tüm metotların sayısıdır. Bu metrik değeri test maliyetinde etkilidir. Bir mesaj çok sayıda metodun çağrılmasını tetikliyorsa, o mesajın ait olduğu sınıfın testi ve hata ayıklaması zorlaşır. Bir sınıftan fazla sayıda metodun çağrılması sınıfın karmaşıklığını artırır.

7.6 Kohezyon (LCOM)

Bağımlılıkla birlikte kullanılan kohezyon değerinin birden fazla tanımı vardır. Chidamber ve Kemerer'de bu metrik, metotların uyumluluğuna, yani metotların kendi içindeki kesişim değerlerine bakar. Bir sınıfın hem metotlarıyla hem de özellikleriyle (yerel değerleriyle) kendi içinde ne kadar uyumlu ve etkin olduğunu belirtir.

Hoca CK dışında MOOD metriği gibi başka metrik kümelerinin de olduğunu belirtti.

> **Ek bilgi (derste anlatılmadı):** CK kümesindeki LCOM'un açılımı büyük olasılıkla "Lack of Cohesion in Methods", yani metotlardaki uyum eksikliğidir. İlk tanımda büyük olasılıkla aynı niteliği paylaşmayan metot çiftlerinin sayısından paylaşan çiftlerin sayısı çıkarılır; sonuç negatifse sıfır alınır. Bu yüzden LCOM değeri büyüdükçe sınıfın uyumu azalır ve sınıfı bölmek gündeme gelir. Daha sonra büyük olasılıkla bu eksiklikleri gidermek için LCOM2, LCOM3 gibi farklı tanımlar önerilmiştir.

8. Dersin O Güne Kadarki Akışı ve Test Tipleri

Yeni konuya geçmeden önce hoca, Yazılım Kalite ve Test Süreci dersinde o güne kadar işlenenleri sıraladı:

- Yazılım mühendisliğinin içinde kalitenin yeri.
- Yazılım kalite özellikleri: Kalite dediğimizde neler anlaşılıyor? McCall'un kalite matrisi ve modeli.
- Kaliteyi sağlama aktiviteleri: Yazılımı geliştirmeye başlamadan önce hangi kalite standartlarına uyulacağını belirlemek; hangi yazılım geliştirme süreç modelinin kullanılacağına karar vermek; formal bir yöntemle, kurallı biçimde geliştirmek; geliştirme sırasında gözden geçirme (review) ve inceleme (inspection) yapmak.
- Test süreci adımları: V model ve bu model içinde doğrulama (verification) ve geçerleme (validation). V model çerçevesinde birim testi, bütünlük testi, regresyon testi, sistem testi, kullanıcı testi, onaylama ve kabul testi. Her adımda test verisinin ve test senaryolarının nasıl hazırlandığı.
- Test tipleri: kara kutu (black box), saydam kutu (white box) ve ikisinin karması olan gri kutu testi.
- Kalite değerlendirmesinin dayanağı olan yazılım metrikleri: Halstead, McCabe karmaşıklığı (program yolları ölçütü), büyüklük ölçütleri (function point ve KLOC) ve CK metrikleri.

Hoca bu noktada öğrencilerin artık standartlar, süreç modelleri ve gözden geçirme ile inceleme konularında soracağı soruları cevaplayabilecek durumda olduklarını söyledi.

8.1 Test Tipleri ile Test Adımları Arasındaki İlişki

Soru: Test süreci adımlarından hangisinde ağırlıklı olarak saydam kutu testi kullanırız?

Cevap: Birim testinde. Hoca, kullanıcı testinde ise yalnızca kara kutu testi yapıldığını ekledi. Kullanıcı kabul işlemini gerçekleştirirken ve denemeler yaparken dışarıdan bakar, iç yapıyla ilgilenmez.

> **Hocanın vurgusu:** Bir yazılım üzerinde ne yalnızca saydam kutu testi ne de yalnızca kara kutu testi yapılır. İkisi birlikte götürülür.

Hoca terim üzerine de durdu: White box testing için İngilizcede glass box testing de kullanılır. Türkçede "saydam kutu" demek daha doğrudur, çünkü iç yapıyı, içini görürüz.

Soru: Yazılım test süreci adımlarında beta testi hangi seviyede gerçekleştirilir ve beta testinde hangi tip test yapılır?

Cevap: Beta testi, kullanıcı testinin bir aşamasıdır. Kullanıcı testinde kara kutu testi yapılır, çünkü kullanıcı iç taraftan değil dış taraftan bakar.

9. Test Yönetimi ve Test Spesifikasyonu

Hoca bu dersin asıl konusunun test yönetimi olduğunu ve bunun dönem projesiyle doğrudan bağlantısını açıkladı. Projede önce gereksinim analizi (requirement analysis) yapılıp gereksinim spesifikasyonu hazırlanacak, ardından test yönetimi gerçekleştirilecektir. Test yönetiminin adımları sırasıyla şunlardır:

- Planlama.
- Tasarlama.
- Testlerin uygulanması, yani çalıştırılması.
- Test raporunun hazırlanması.

Bu bölümün hedefleri: test spesifikasyonu belgesinin içeriğini incelemek, test planının içeriğini öğrenmek ve test senaryosu hazırlamak.

Gereksinim raporu hazırlandığında yazılımdaki bütün temel işlemler açıklanmış olur. Sonraki sorular şunlardır: Bu işlemlerin her birini nasıl test ederiz? Test planı ne olacak? Bu plan çerçevesinde hangi test verisiyle hangi testler çalıştırılacak? Testler çalıştırılırken hangi aşamalardan geçildi? Bunların hepsi raporlanır.

Hoca test planını, Yazılım Mühendisliği dersinde projeye başlamadan önce oluşturulan proje planına benzetti. Burada da teste başlamadan önce bir test planı oluşturulur: Hangi testlerin yapılacağına karar verilir, hangi tip testlerin uygulanacağı belirlenir ve aşamalar bir Gantt diyagramı çerçevesinde zaman planına bağlanır.

9.1 Test Spesifikasyonunun Bölümleri

Test kapsamında yapılan işlemler planlama, tasarım ve gerçekleştirme adımlarıyla yürütülür. Yazılım testlerini tanımlamak, planlamak, düzenlemek ve belgelemek için test spesifikasyonu adı verilen bir belge düzenlenir. Hocaya göre test spesifikasyonu dediğimizde test planını, test verisi olarak seçilen test senaryolarını, testlerin nasıl çalıştırıldığını ve test raporunu içine alan bütün anlaşılır. Bölümler şöyledir:

- **Test planı:** Hangi tip testler yapılacak, zamanlaması nasıl olacak, gideri ve ortamı ne olacak?
- **Test senaryoları:** Hangi testler için hangi test verisi kullanılacak?
- **Test işlemleri:** Her testin tanımı, test edilen modüller ve kullanılan özel araç ve teknikler.
- **Beklenen sonuç:** Gereksinim analizinden çıkarılır.
- **Gerçek test sonucu:** Test edildiğinde çıkan değer.

Hoca test işlemleri için bir örnek verdi: Java ile yazılmış bir programın birim testi anlatılacaksa, raporda birim testi yapıldığı, Java'da yazılan kodun JUnit ile çalıştırıldığı ve hangi modülün test edildiği yazılır. Özel araç ve teknikler kısmında JUnit kullanıldığı belirtilir.

Proje kapsamında hoca bazı sadeleştirmelere izin verdi:

- Gider kısmına şu an bir şey yazılmasa da olur.
- Test senaryoları ile test işlemleri birleştirilebilir. Test senaryolarına ayrı bir madde açmak gerekmez; her çalıştırılan testte hangi senaryonun kullanıldığı belirtilir.
- İsteyen ayrıca özet bir sonuç raporu yazabilir.

  Hocanın vurgusu: Raporda mutlaka olması gerekenler şunlardır: test planı, test işlemlerinin tek tek nasıl gerçekleştirildiğini açıklayan kısım ve beklenen sonuçlarla gerçek test sonuçlarının kıyaslanması.

9.2 Test Yönetimi Süreci ve Standart

Hoca slaytta test adımlarını gösterdi: Önce test metodolojisi belirlenir, ardından testin planı, testin tasarımı ve testin gerçekleştirilmesi gelir. Test yönetimi geri beslemeli bir süreçtir. Hata mesajları da burada belirtilir: Hangi hatalar alındı? Hata mesajının içeriği de belge olarak verilebilir.

Testin spesifikasyonunu tanımlayan standart, IEEE standartları içindeki yazılım test belgelendirme (Software Test Documentation) standardıdır. Gereksinim spesifikasyonunun içindekiler listesini nasıl bir standart tanımlıyorsa, test spesifikasyonunun içeriğini de bu standart tanımlar.

> **Ders sonrası not:** Hoca slayttaki numaranın hatalı olduğunu kendisi söyledi: 830, gereksinim spesifikasyonu standardının numarasıdır (IEEE 830). Yazılım test belgelendirme standardı büyük olasılıkla IEEE 829'dur. Bu standart büyük olasılıkla 1983'te yayımlanmış, 1998 ve 2008'de güncellenmiştir. 2008 sürümü 2013'ten sonra büyük olasılıkla yerini ISO/IEC/IEEE 29119-3'e bırakmıştır.

9.3 Test Spesifikasyonunun Geliştirilmesi

Test spesifikasyonu bir döngüyle geliştirilir: Önce yazılır, incelenir, gerekirse düzeltilir ve tekrar üzerinde çalışılarak devam edilir. Hoca, ileride konfigürasyon yönetiminde görülecek temel çizgi (baseline) kavramının burada da geçerli olduğunu belirtti: Hangi aşamadaki test tanımının kaçıncı sürüm olduğu izlenir.

10. Test Planının İçeriği

Hoca projede hazırlanacak test planının içeriğini slayttan sıraladı:

- **Test stratejisi ve test edilecek öğeler:** Test stratejisi ne olacak, yazılım hangi test adımlarından geçirilecek? Test edilecek elemanlar nelerdir? Bu kısım yazılımın gereksinim raporuna atıf yapılarak yazılır.
- **Doğrulama yöntemleri:** Doğrulama ve geçerleme yöntemi olarak ne kullanıldı?
- **Testlerin tamamlanma kriterleri:** Örneğin hata oranı hangi değere gelince test durdurulacak?
- **Hata ve test sonuç raporlama adımları.**
- **Test sorumlulukları:** Bir ekip mi kuruldu? Nasıl bir ekip yapısı içinde çalışılacak? Hangi görevler var?
- **Test ortamı:** Hangi test yazılım araçları kullanılacak?
- **Eleman ihtiyacı:** Kaç testçiye (tester) ihtiyaç var? Bunlara eğitim verilmesi gerekiyor mu?
- **Test takvimi:** Örneğin bir Gantt diyagramı.
- **Risk yönetimi:** Test sürecinde bir şey olursa risk nasıl yönetilecek?
- **Test çıktıları:** Testten beklenen çıktılar neler?

Hoca slaytta biraz fazla ayrıntı verdiğini, plandaki ayrıntıların değişebileceğini ve standartlar dikkate alınarak hazırlanacak bir test planının daha az maddeden oluşabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** IEEE 829'un test planı şablonu büyük olasılıkla şu başlıkları içerir: test planı kimliği, giriş, test edilecek öğeler, test edilecek ve edilmeyecek özellikler, yaklaşım, öğe geçti/kaldı ölçütleri, askıya alma ve devam ettirme ölçütleri, test çıktıları, test görevleri, ortam gereksinimleri, sorumluluklar, eleman ve eğitim ihtiyacı, takvim, riskler ve olası durumlar, onaylar. Hocanın slayttaki maddelerinin çoğu bu başlıklarla örtüşür. Aynı standart test planı dışında büyük olasılıkla test tasarım, test durumu ve test prosedürü belirtimlerini, test olay raporunu ve test özet raporunu da tanımlar.

11. Test Senaryosu

Bir test senaryosunda şunlar bulunur:

- Senaryonun adı ve kimliği.
- Senaryonun hangi gereksinimleri test etmek için hazırlandığı.
- Testin amacı.
- Ön koşullar, varsa.
- Varsayımlar, varsa.
- Test girdileri.
- Test senaryosunun adımları.
- Beklenen sonuçlar.

Senaryonun nasıl hazırlanacağı, testin hangi seviyede ve hangi tipte yapıldığına bağlıdır:

- **Saydam kutu testi yapılırken (birim testi veya bütünlük testi):** Senaryo yalnızca iç yapıyı test etmek ya da tasarımın ayrıntılarını bilerek tasarıma uygun bir bütünleme yapılıp yapılmadığını kontrol etmek için hazırlanır.
- **Kullanıcı testi bazında, geçerlemede (validation):** Yazılımın doğruluğu dışarıdan bakılarak değerlendirilir. Bu durumda senaryo dışarıdaki kullanıcının görüşüne göre hazırlanır.

Hoca kullanıcı senaryosu için bir öğrenci bilgi sistemi örneği verdi. Öğrenci işleri daire başkanlığında çalışanlar kendi birimleriyle ilgili alt işlemleri değerlendirirken senaryolar onlarla birlikte oluşturulur. Örneğin şu durumdaki bir öğrencinin ders seçimi, belli bir gruptaki öğrencinin (mesela çift anadal yapan öğrencinin) ders seçimi ya da sınav değerlendirmesi gibi adımlar kullanıcıyla birlikte belirlenir. Senaryo bu şekilde farklı girdiler için oluşturulur ve denemeler yapılır.

12. Test Çalışma Ekibi ve Optimum Test Miktarı

12.1 Test Ekibi

Yazılımın test edilmesi sürecinde teste bağımsızlık özelliğinin sağlanması önemlidir. Yazılımın test edilmesinden sorumlu olan kişiler, yazılımı tasarlayan ve oluşturan grubun içinde olma-

malıdır. Hocaya göre dışarıdan bir kişi daha iyi bakabilir.

Test ekibinin görevleri şunlardır:

- Planlarda belirtilen testler için gerekli test hazırlığını yapmak.
- Testleri yürütmek.
- Testlerde bulunan hataları yazılımcıya bildirmek ve izlemek.
- Test sonuçlarını raporlamak.

12.2 Optimum Test Miktarı

> **Hocanın vurgusu:** İyi bir testçi mükemmelliğe ne zaman ulaşılamayacağını ve neyin yeterli olacağını bilir. Her yazılım projesinde optimum test miktarı farklıdır.

Test sayısında her zaman bir denge gözetilir. Bir yandan kaçırılan hata (bug) sayısını en aza indirmek hedeflenir, öbür yandan test maliyetini çok yukarı çıkarmamak hedeflenir. Hoca bunu slayttaki bir grafik üzerinde anlattı:

- Mavi eğri test maliyetidir; test miktarı arttıkça yükselir.
- Kırmızı eğri kaçırılan hata sayısıdır; test miktarı arttıkça düşer. Bu sayı mümkün olduğunca aşağı çekilmek istenir.
- İki eğrinin kesişim noktası optimum test miktarını verir. Bu noktadan sonrası test dışı bırakılır.

13. Yazılım Yaşam Döngüsü Boyunca Test Etkinlikleri

Hoca aradan sonra yazılım yaşam döngüsü boyunca sınama (test) etkinliklerini gösteren slayta geçti. Her aşamada test adına yapılan işler farklıdır:

Aşama	Test etkinliği
Planlama	Sistemin test planı, kalite planı
Gereksinim analizi	Her alt gereksinimin nasıl test edileceğine dair test planları
Tasarım	Modül test planı
Gerçekleme	Birim testi, bütünlük testi, sistem testi
Kurulum	Kullanıcı testi, test raporlarının tamamlanması

Aşamaların açıklaması şöyledir:

- **Planlama:** Yazılımın planlanması sırasında sistemin test planı ve kalite planı oluşturulur.
- **Gereksinim analizi:** Her bir alt gereksinimin nasıl test edileceğine dair test planları bir yandan oluşturulabilir.
- **Tasarım:** Modül için test planı yapılır: Modül nasıl bir veriyle test edilebilir? Tasarım sırasında McCabe karmaşıklığıyla ilgili algoritma yolları belli olur. Her modülün tasarımı ve algoritması yapılırken bu algoritmanın nasıl test edileceği de planlanabilir.
- **Gerçekleme:** Her modül kodlandıktan sonra birim testi yapılır. Bütün modüller bitince bütünlük testi tamamlanabilir; her alt modül test edildikten sonra bütün de test edilebilir. Hoca sistem testinin modül bazında da yapılabileceğini daha önce söylemişti; bu yüzden gerçekleme sırasında bir yandan sistem testi de yapılabilir.
- **Kurulum:** Yazılım artık kullanıcıya teslim edilmiştir. Kullanıcı testi yapılır ve test raporları bu aşamada tamamlanır.

Bu adımlar seçilen yazılım geliştirme süreç modeline göre değişir. Bazı modellerde test planları daha ilk çalışmalar başladığında oluşturulur, bazılarında ise geliştirme sürecinin sonlarına doğru yapılır. Hoca bu notu tekrar vurguladı: Süreç modeline bağlı olarak her adımda test adına yapılan işler artabilir ya da azalabilir.

Aradan önce hoca, bir öğrencinin sorusu üzerine sistem testine dair bir yönlendirme de yaptı: Seçilen yazılımla ilgili hangi sistem testi ortamlarının bulunduğu araştırılmalı ve performans

testi, güvenlik testi ya da dayanıklılık testi gibi test yapan araçlardan biri (tool) kullanılmalıdır. Dersin sonunda bunu, projede kullanılan dilin birim testi olanaklarını araştırma isteğiyle birlikte tekrarladı.

14. Teslim Sonrası Kalite: Yazılım Bakımı

Hoca kaliteyi sağlama aktivitelerini, ilk derste de yaptığı gibi yeniden sıraladı: standartların belirlenmesi, yazılım geliştirme süreç modelinin belirlenmesi, geliştirme sırasında gözden geçirme, inceleme ve test, yazılım metrikleriyle büyüklük ölçümü. Peki yazılım teslim edilip kurulumu yapıldıktan sonra kalite nasıl sağlanır? Bunun için iki etkinlik vardır:

- Bakım (maintenance) aşaması: yazılımın korunması, onarılması ve geliştirilmesi.
- Konfigürasyon yönetimi: Teslimden sonra sürüm değişikliği yapıldığında da yazılımın bütün elemanlarıyla ilgili en son bilginin tutulması ve saklanması.

14.1 Bakım Nedir ve Bakım İşlerinin Dağılımı

Yazılım bakımı ve onarımı (software maintenance), sonradan görülen hataların düzeltilmesi, yazılımın iyileştirilmesi, uyarlanması ve geliştirilmesi olarak görülebilir. Hoca bakım işlerinin dağılımını şöyle verdi:

Bakım işi	Pay
Hata düzeltme	%21
İyileştirme	%25
Uyarlama	%50
Diğer	%4

Oranların toplamı $21 + 25 + 50 + 4 = 100$'dür.

- **Hata düzeltme:** Bir hatayla karşılaşılır ve düzeltilir.
- **İyileştirme:** Yeni ve gelişmiş yazılım araçlarıyla yazılımın kalitesinin iyileştirilmesi, yenilikler yapılması.
- **Uyarlama:** Yazılımın kullanıldığı ortamda, yani gerçek hayattaki uygulamada değişiklik olmuşsa bunun yazılıma yansıtılmasıdır.

  Ders sonrası not: Bakım dağılımı için yaygın kaynak olan Lientz ve Swanson çalışmasında oranlar büyük olasılıkla şöyledir: iyileştirme (perfective) yaklaşık %50, uyarlama (adaptive) yaklaşık %25, hata düzeltme (corrective) yaklaşık %21, diğer (preventive vb.) yaklaşık %4. Hocanın derste söylediği oranlarda iyileştirme ile uyarlamanın yerleri büyük olasılıkla karışmıştır.

  Ek bilgi (derste anlatılmadı): Bakım türleri literatürde büyük olasılıkla genelde dört sınıfa ayrılır: düzeltici bakım (corrective, hata düzeltme), uyarlayıcı bakım (adaptive, ortam değişikliğine uyum), iyileştirici bakım (perfective, yeni özellik ve performans iyileştirmesi) ve önleyici bakım (preventive, ileride çıkabilecek sorunları önlemek için yeniden yapılandırma). Hocanın "diğer" diye andığı küçük pay büyük olasılıkla önleyici bakıma karşılık gelir. Bu sınıflandırma büyük olasılıkla ISO/IEC 14764 bakım standardında da kullanılır.

14.2 Bakımda Karşılaşılan Durumlar

Hoca bakım sırasında yapılan işleri üç durumla açıkladı:

- Büyük bir yazılımda test aşamasında bulunamamış çeşitli hatalar sonradan ortaya çıkabilir. Bunlar işlem hatası, yetenek (performans) hatası ya da tasarım hatası olabilir. Ne kadar hatasız teslim etmeye çalışırsak çalışalım, sonradan hata çıkabilir.
- Yazılımın yeteneğini iyileştirmek ve kullanımını kolaylaştırmak için programa bazı eklemeler gerekebilir. Bu durumda birkaç modülde değişiklik yapmak gerekir.
- Günün koşullarına göre yeterli bulunmayan bir yazılım, gereksinim analizi, tasarım ve test adımlarıyla yeniden geliştirilebilir.

14.3 Bakım Maliyetini Azaltmak

Yazılımı geliştirirken harcanan emeğin bir maliyeti vardır. Ama teslimden sonra bir hata ortaya çıkarsa onu düzeltmek ve bakım sözleşmeleri kullanıcılar için çok daha maliyetli olur. Yazılım geliştiriciler açısından da yazılım hatasından kaynaklanan bir sorun, geliştiriciye duyulan güveni sarsan ayrı bir maliyet olarak karşımıza çıkar.

Bakım aşamasındaki gideri azaltmak için yazılımın geliştirme sırasında bakım ve onarıma elverişli olması, yani bakım kolaylığı (maintainability) özelliğine sahip olması gerekir. Bunun için:

- Yazılım mühendislerinin daha dikkatli ve deneyimli olması.
- Sistemin daha anlaşılabilir bir yapıda olması.
- Standart programlama dilleri ve işletim sistemleri kullanmak.
- Belgeleri standartlara uygun hazırlamak.
- Test programlarından ve test araçları gibi yazılım araçlarından faydalanmak.
- Hataların tasarım aşamasında bulunabilmesi için tüm gayreti göstermek.

Yazılımın bakım ve onarıma elverişliliği şu kalite faktörleriyle ortaya çıkar:

- **Test edilebilirlik:** Test işlemini kolaylaştıracak özelliklerde olması.
- **Basitlik:** Daha basit bir tasarımla hazırlanmış olması.
- **Değiştirilebilirlik:** Hocaya göre değiştirilebilirliğin altında modülerlik yatar.
- **Taşınabilirlik:** Bir işletim sisteminden başka bir işletim sistemine aktarılabilme özelliği.
- **Güvenirlik.**
- **Esneklik:** Esnek bir yapıda hazırlanmış olması.

15. Yazılım Konfigürasyon Yönetimi

Bakımda yapılan her değişiklikte ve geliştirme sırasında başlayan özelliklerle birlikte yazılım konfigürasyon yönetiminin sağlanması gerekir. Hoca bu konuyu Yazılım Mühendisliği dersinden bilinmesi gereken bir konu olarak hatırlattı.

Yazılım mühendisliği ürünleri, yani yazılımın elemanları programlar, belgeler ve veri yapılarıdır; ayrıca veritabanı içindeki elemanlar da bunlara dahildir. Bu ürünlere ait bütün maddelere topluca yazılım konfigürasyonu denir. Başka bir deyişle yazılımın düzeni ya da yapısıdır.

Yazılım konfigürasyon maddesi (Software Configuration Item, SCI), konfigürasyon yönetimi işlemlerinin uygulandığı tek tek yazılım elemanlarıdır.

Yazılım düzeni yönetiminde yapılan işlemler şunlardır:

- **Konfigürasyon tanımlama (configuration identification):** Bütün elemanların tek tek tanımlanması. Her modülün, veritabanındaki her elemanın, belgelerin ve test senaryolarının tanımlanması gibi.
- **Konfigürasyon değişiklik kontrolü.**
- **Konfigürasyon raporlama.**

Konfigürasyon yönetiminde yapılacak her işlem standartlarda tanımlanmıştır. Örneğin yazılım konfigürasyon yönetim planının nasıl olması ve neleri içermesi gerektiğini belirten bir standart vardır. Hoca ISO 9004'te de konfigürasyon yönetimiyle ilgili bir tanımlama bulunabileceğini söyledi.

> **Ders sonrası not:** Konfigürasyon yönetim planı için daha doğrudan kaynak büyük olasılıkla IEEE 828 (Software Configuration Management Plans), genel konfigürasyon yönetimi rehberi için ISO 10007'dir. ISO 9004 genel bir kalite yönetimi rehberidir. Konfigürasyon yönetiminin klasik dört işlemi büyük olasılıkla tanımlama, değişiklik kontrolü, durum raporlama (status accounting) ve denetimdir (audit).

Konfigürasyon yönetimiyle birlikte sürüm yönetimine de bakılır. Sürüm yayın yönetimi (release management) kavramı da konfigürasyon yönetimine destek sağlar. Hoca, konfigürasyon yönetiminin ve özellikle sektörde önem kazanan release management kavramının, ikisi arasındaki ilişkiyle birlikte sonraki derslerde ayrıntılı işleneceğini söyledi.

Örnek Sorular

Bu dersin video kaydı ve ekran karesi yoktur. Aşağıdaki sorular, hocanın derste sözlü sorduğu sorulardır.

Soru 1 (derste sözlü, 00:08:09): Bir yazılımın kodlaması tamamlandı. Test süreci adımlarında sırasıyla hangi testler gerçekleştirilir?

> **Çözüm (derste verildi, 00:09:33):** Birim testi (unit test), bütünlük testi (integration test), sistem testi, kullanıcı testi. Test planlaması ve tasarımı bu sorunun cevabı değildir; onlar test yönetiminin adımlarıdır.

Soru 2 (derste sözlü, 00:28:27 ve 00:29:55): Bir yazılımın büyüklüğünü hangi yollarla ölçebiliriz? Veritabanı (dosyalar), girdi-çıktı değerleri ve ara birim değerleri sayılıp bunlar üzerinden hesaplama yapılan dolaylı büyüklük ölçümünün adı nedir?

> **Çözüm (derste verildi, 00:30:50-00:31:44):** Dolaylı ölçüm işlev noktasıdır (function point); bir karmaşıklık değil, büyüklük ölçüsüdür. Doğrudan ölçüm ise kod satırı sayısıdır (LOC, KLOC).

Soru 3 (derste sözlü, 00:32:14): Kod satırı (LOC) ile büyüklük ölçmenin kötü tarafları, dezavantajları nelerdir?

> **Çözüm (derste verildi, 00:32:50-00:35:19):** Yazım biçimi satır sayısını değiştirir (ayrı satırlara yazılan parantezler gibi). Aynı iş farklı dillerde farklı satır tutar (C, Assembler); bunun için diller arası eşleştirme tabloları hazırlanmıştır. Daha iyi bir algoritma ve yeniden kullanım satır sayısını düşürür. Bu yüzden emek satır sayısıyla her zaman doğru orantılı değildir. Yine de ölçmesi kolay olduğu için kullanılır.

Soru 4 (derste sözlü, 01:13:50): Test süreci adımlarından hangisinde ağırlıklı olarak saydam kutu testi kullanırız?

> **Çözüm (derste verildi, 01:14:26-01:15:25):** Birim testinde. Kullanıcı testinde ise yalnızca kara kutu testi yapılır. Bir yazılımda iki tip test birlikte yürütülür.

Soru 5 (derste sözlü, 01:16:05): Yazılım test süreci adımlarında beta testi hangi seviyede gerçekleştirilir ve beta testinde hangi tip test yapılır?

> **Çözüm (derste verildi, 01:17:18-01:17:30):** Beta testi kullanıcı testinin bir aşamasıdır. Kara kutu testi olarak yapılır, çünkü kullanıcı iç yapıya değil dışarıdan bakar.

Dersin Toparlaması

Dersin ilk yarısı, test yönetimine zemin hazırlayan yazılım ölçümü konusunun tekrarıydı. Ölçüm, yazılım sürecini yönetmek, kaliteyi izlemek ve sonraki projeler için arşiv oluşturmak için gereklidir. Halstead metriği işleç ve işlenenlere, McCabe metriği akış grafındaki bölgelere ve bağımsız yollara dayanır; McCabe değerinin 10'u aşması modülün bölünmesi gerektiğine işaret eder. Güvenirlik geçmiş istatistiklere dayalı bir kestirimdir. Büyüklük doğrudan (LOC) ya da dolaylı (function point) ölçülür; LOC'un dezavantajları bilinse de kolaylığı nedeniyle kullanılır. CK metrikleri nesneye yönelik tasarımda karmaşıklığı, kalıtımı, bağımlılığı ve uyumu ölçer ve çoğu doğrudan test maliyetiyle ilişkilidir.

İkinci yarıda test yönetimi işlendi: planlama, tasarım, uygulama ve raporlama adımları; test planı, test senaryoları, test işlemleri ve beklenen ile gerçek sonuçların karşılaştırılmasından oluşan test spesifikasyonu; test planının içeriği; senaryoların test seviyesine göre içeriden ya da kullanıcı gözünden hazırlanması; test ekibinin bağımsızlığı ve maliyet ile kaçırılan hata arasındaki dengeden doğan optimum test miktarı. Ardından test etkinliklerinin yaşam döngüsüne nasıl dağıldığı ve teslim sonrası kalitenin iki ayağı, yazılım bakımı ve konfigürasyon yönetimi görüldü. Hoca konfigürasyon yönetimi ile sürüm yönetiminin (release management) sonraki derslerde ayrıntılı işleneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.